VECTOR INDIA

(Advanced Embedded System Training Institute)

[Bangaluru, Chennai, Hyderabad]

A PROJECT SYNOPSIS

ON

Smart Healthcare Monitoring and Emergency Assistance System

An Embedded Industrial Safety Monitoring System Based on the LPC2129 ARM7 Microcontroller

---

SUBMITTED BY (GROUP NO : BA3)

1. Sai Suvarna (V25BE7E2)
2. Deepak Kumar (V25BE7D2)
3. Vaibhav Hinchageri (V25BE7V3)
4. Venkateshwara Surendra (V25BE7V7)

VECTOR INDIA INSTITUTE
(33/49,12th Main Rd,4th T Block East, KV Layout, Jayanagar, Bengaluru, Karnataka 560011)
[July 2026]

INDEX

CHAPTER 1 | INTRODUCTION

1.1 Background and Need for the Project

The healthcare sector requires continuous monitoring of patients to ensure timely medical assistance during critical situations. In hospitals, clinics, and home healthcare environments, manually monitoring vital signs such as body temperature, heart rate, blood oxygen saturation (SpO_2), and saline level is difficult and time-consuming. Delays in detecting abnormal health conditions can lead to serious complications and may put the patient's life at risk. Therefore, an automated monitoring system is essential to improve patient safety and support healthcare professionals.

The **Smart Healthcare Monitoring and Emergency Assistance System** is developed using the **LPC2129 ARM7 microcontroller** to continuously monitor the patient's health parameters in real time. The system uses the **DHT11** sensor to measure body temperature, the **MAX30102** sensor to monitor heart rate and SpO_2, and the **HC-SR04** sensor to detect saline level. The **DS1307 RTC** provides accurate date and time information, while the patient's health status is displayed on a **16×2 LCD**.

Whenever an abnormal condition is detected or the emergency switch is pressed, the system activates the **buzzer** and **LED indicators** and sends an alert to the caregiver through the **HC-05 Bluetooth module**. This embedded healthcare system provides a reliable, low-cost, and efficient solution for continuous patient monitoring in hospitals, clinics, and home healthcare, enabling faster medical response and improving overall patient care.

1.2 Problem Statement

Hospitals and home healthcare environments often rely on manual observation and periodic measurement of a patient's vital signs, making continuous monitoring difficult. This approach is time-consuming, inconsistent, and reactive rather than preventive, as critical changes in body temperature, heart rate, blood oxygen saturation (SpO_2), or saline level may go unnoticed until the patient's condition becomes serious. Delayed detection of abnormal health conditions can lead to slower medical response and increased risk to patient safety.

There is a clear requirement for an embedded, low-cost, real-time patient monitoring system capable of continuously observing multiple vital health parameters simultaneously, immediately identifying abnormal conditions, and triggering appropriate visual, audible, and wireless alerts without depending solely on continuous human supervision. Such a system can improve patient safety, support healthcare professionals, and enable faster emergency medical assistance in hospitals, clinics, and home healthcare environments.

1.3 Objectives of the Project

- To design and develop a **Smart Healthcare Monitoring and Emergency Assistance System** using the **LPC2129 ARM7 microcontroller**.
- To continuously monitor the patient's **body temperature, heart rate, blood oxygen saturation (SpO_2), and saline level** in real time.

- To detect abnormal health conditions by comparing the measured parameters with predefined threshold values.
- To display the patient's health status and alert messages on a **16×2 LCD**.
- To provide immediate visual and audible alerts using **LED indicators** and a **buzzer** whenever an abnormal condition is detected.
- To transmit emergency notifications to caregivers through the **HC-05 Bluetooth module** for faster medical assistance.
- To provide accurate **date and time information** using the **DS1307 RTC** module.
- To include an **Emergency Push Button** that allows the patient or caregiver to manually trigger an emergency alert during critical situations.
- To develop a **low-cost, reliable, and real-time embedded healthcare system** suitable for hospitals, clinics, and home healthcare applications.

1.4 Scope of the Project

The **Smart Healthcare Monitoring and Emergency Assistance System** is designed for hospitals, clinics, and home healthcare environments where continuous patient monitoring is essential. The system continuously measures body temperature, heart rate, blood oxygen saturation (SpO_2), and saline level in real time using embedded sensors connected to the LPC2129 ARM7 microcontroller. It provides immediate visual, audible, and Bluetooth-based alerts whenever abnormal health conditions are detected or the emergency switch is pressed, enabling faster medical assistance. This project demonstrates a reliable, low-cost, and practical embedded healthcare solution that improves patient safety and supports efficient patient monitoring in various healthcare applications.

CHAPTER 2 | COMPONENTS USED

2.1 Hardware Components

The project uses a combination of sensing, processing, communication and output devices, organised into three functional groups as shown below.

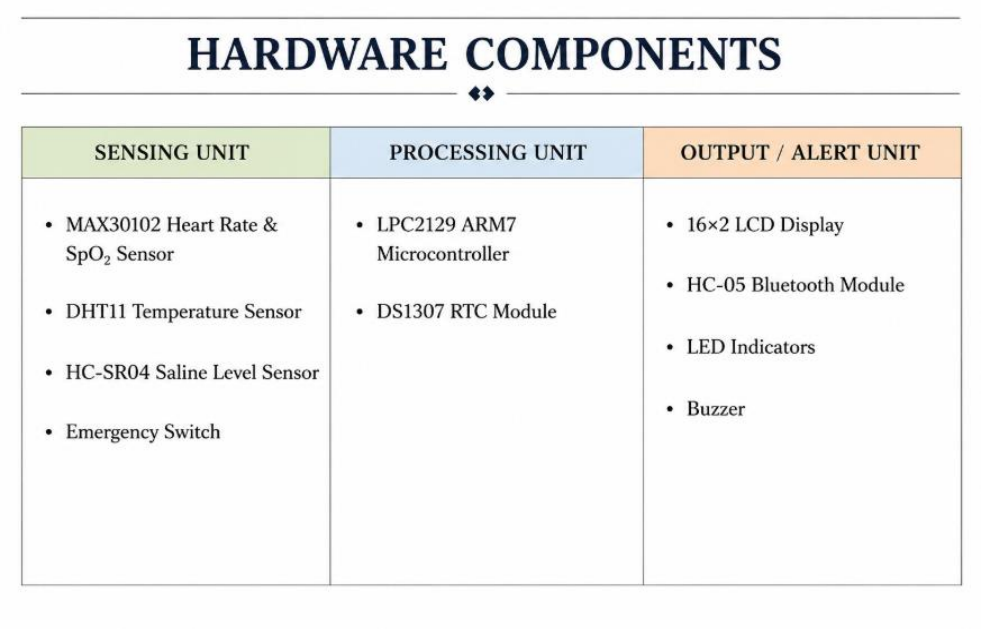

Fig. 2.1 – Component Category Overview

Sl. No.	Component Used	Function of the Component
1.	LPC2129 ARM7 Microcontroller	Controls the entire system and processes sensor data.
2.	MAX30102 Heart Rate & SpO_2 Sensor	Measures heart rate and blood oxygen saturation.
3.	DHT11 Temperature Sensor	Measures the body temperature.
4.	HC-SR04 Saline Level Sensor	Measures the saline level in the IV fluid bottle.
5.	16×2 LCD Display	Displays all health parameters and system messages.
6.	HC-05 Bluetooth Module	Sends health data and alerts to the caregiver.
7.	LED Indicators	Shows normal or abnormal status visually.
8.	Buzzer	Provides an audible alert in emergencies.
9.	DS1307 RTC Module	Provides real-time date and time.
10.	Emergency Switch	Used to manually trigger an emergency alert.

2.2 Software Tools

- **Keil µVision IDE-**Used to write, compile, and debug the Embedded C program for the **LPC2129 ARM7 microcontroller** in this project.
- **Embedded C -** Used to develop the program that reads sensor data, processes it, and controls the LCD, buzzer, LEDs, and Bluetooth module.
- **Flash Magic-** Used to upload the compiled **HEX** file into the **LPC2129 microcontroller** for execution.

2.3 Communication Interfaces Used

Sl. No.	Interface Used	Usage in the Project
1.	I^2C (Inter-Integrated Circuit)	Used for two-way serial communication to read heart rate and SpO_2 data from MAX30102 sensor and for data storage in EEPROM, and to communicate with RTC module to get real-time date and time.
2.	GPIO (General Purpose Input/Output)	Used to receive data from DHT11 temperature sensor and HC-SR04 saline level sensor, and to control LED indicators and buzzer.
3.	UART (Universal Asynchronous Receiver Transmitter)	Used to transmit patient data and alerts wirelessly to mobile app via Bluetooth (HC-05 module).

CHAPTER 3 | WORKING AND BLOCK DIAGRAM EXPLANATION

3.1 System Block Diagram

The **System Block Diagram** illustrates the overall architecture of the **Smart Healthcare Monitoring and Emergency Assistance System**. The **LPC2129 ARM7 microcontroller** acts as the central processing unit and interfaces with the **MAX30102 heart rate and SpO_2 sensor**, **DHT11 temperature sensor**, **HC-SR04 saline level sensor**, **DS1307 RTC module**, and the **Emergency Switch**. It continuously acquires sensor data and processes the patient's health parameters in real time. The microcontroller also coordinates communication between all hardware modules to ensure smooth system operation.

The processed data is displayed on the **16×2 LCD** for easy monitoring, allowing caregivers to observe the patient's condition continuously. If any abnormal condition is detected or the emergency switch is pressed, the microcontroller immediately activates the **LED indicators** and **buzzer** to provide local alerts. Simultaneously, the **HC-05 Bluetooth module** transmits an emergency notification and patient status to a connected mobile device. The **DS1307 RTC module** provides accurate date and time information for monitoring purposes. This integrated architecture ensures reliable, real-time patient monitoring and enables quick medical assistance during emergency situations.

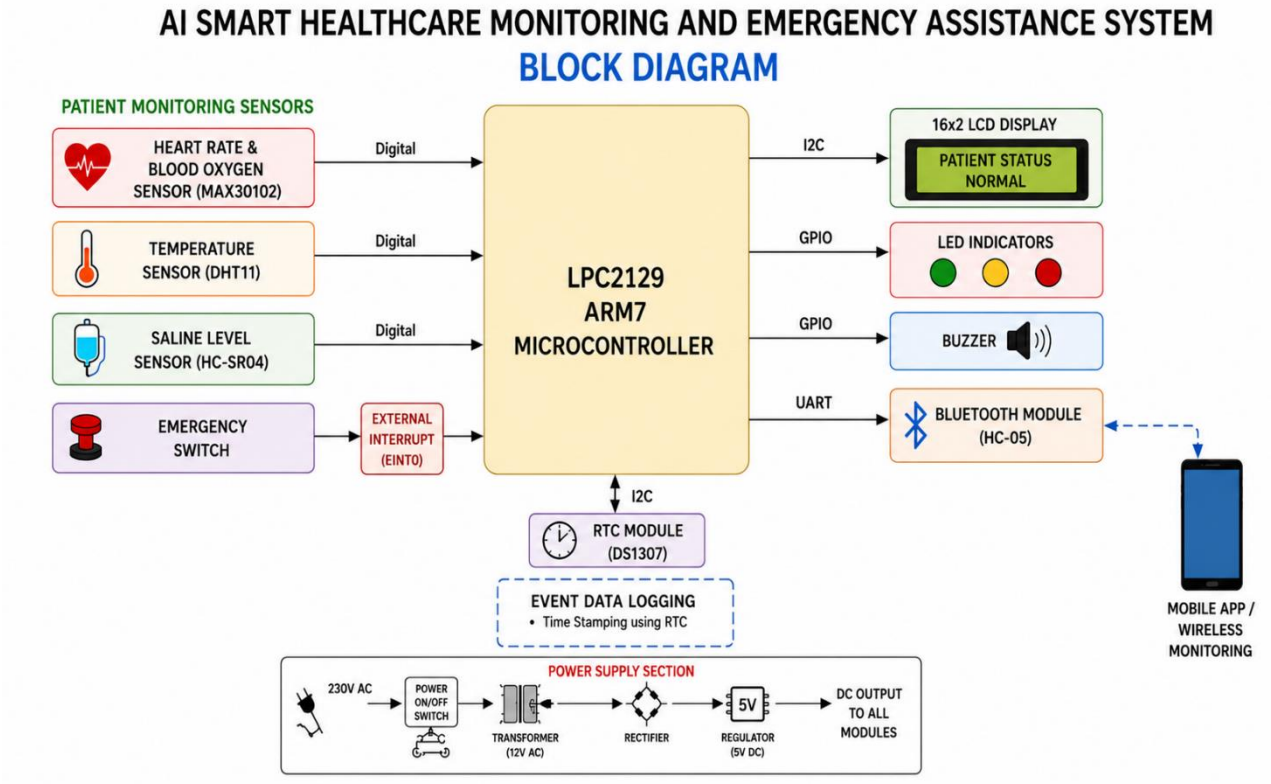

Fig. 3.1 – Block Diagram

3.2 Stage-wise Working Principle

The working principle of the proposed system is divided into different stages to explain the sequence of operations. Each stage performs a specific function, from sensor initialization and data acquisition to processing, alert generation, and wireless communication.

Stage	Stage Name	Working Principle
Stage 1	System Initialization	The LPC2129 microcontroller initializes all sensors (MAX30102, DHT11, HC-SR04), LCD, Bluetooth module (HC-05), RTC (DS1307), LEDs, buzzer, and communication interfaces (I2C, GPIO, UART).
Stage 2	Data Acquisition	The MAX30102 sensor measures heart rate and SpO_2, the DHT11 sensor measures body temperature, and the HC-SR04 sensor measures saline level. The Emergency Switch can be pressed by the patient in critical situations.
Stage 3	Data Processing	The microcontroller reads the sensor data, processes it, and compares the values with predefined safe threshold limits to determine the patient's health status.
Stage 4	Status Display	The processed health parameters and system status are displayed on the 16×2 LCD in real time.
Stage 5	Alert Generation	If any parameter exceeds the safe threshold or the emergency switch is pressed, the buzzer sounds and the LEDs (green/yellow/red) indicate the severity of the condition.
Stage 6	Wireless Communication	The HC-05 Bluetooth module (via UART) sends the patient's health data and alert messages to the caregiver's mobile device for timely medical assistance.

3.3 Working Flow Diagram

The **Working Flow Diagram** illustrates the sequence of operations performed by the **Smart Healthcare Monitoring and Emergency Assistance System**. It represents the complete workflow of the system, starting from initialization and continuing through continuous patient monitoring, data processing, alert generation, and wireless communication. The flow diagram helps in understanding the logical execution of each stage in the embedded system.

Initially, the **LPC2129 ARM7 microcontroller** initializes all connected peripherals, including the **MAX30102 heart rate and SpO_2 sensor**, **DHT11 temperature sensor**, **HC-SR04 saline level sensor**, **DS1307 RTC module**, **16×2 LCD**, **HC-05 Bluetooth module**, **LED indicators**, **buzzer**, and the **Emergency Push Button**. Once initialization is complete, the system enters continuous monitoring mode.

The microcontroller periodically acquires sensor data from all connected sensors. The DHT11 measures the patient's body temperature, the MAX30102 measures heart rate and blood oxygen saturation (SpO_2), and the HC-SR04 monitors the saline level. The DS1307 RTC provides accurate date and time information during system operation.

After acquiring the sensor readings, the microcontroller processes the data and compares each parameter with predefined threshold values. If all the parameters remain within the safe operating range, the current health status is displayed on the **16×2 LCD**, and the monitoring cycle continues without interruption. This continuous monitoring loop ensures that the patient's condition is updated in real time.

Whenever an abnormal health condition is detected or the **Emergency Push Button** is pressed, the microcontroller immediately activates the **LED indicators** and **buzzer** to provide local visual and audible alerts. At the same time, the patient's status and emergency notification are transmitted through the **HC-05 Bluetooth module** to a connected mobile device, enabling caregivers to respond quickly. After completing the alert process, the system automatically returns to the monitoring state and continues checking the patient's vital parameters, ensuring reliable and uninterrupted healthcare monitoring.

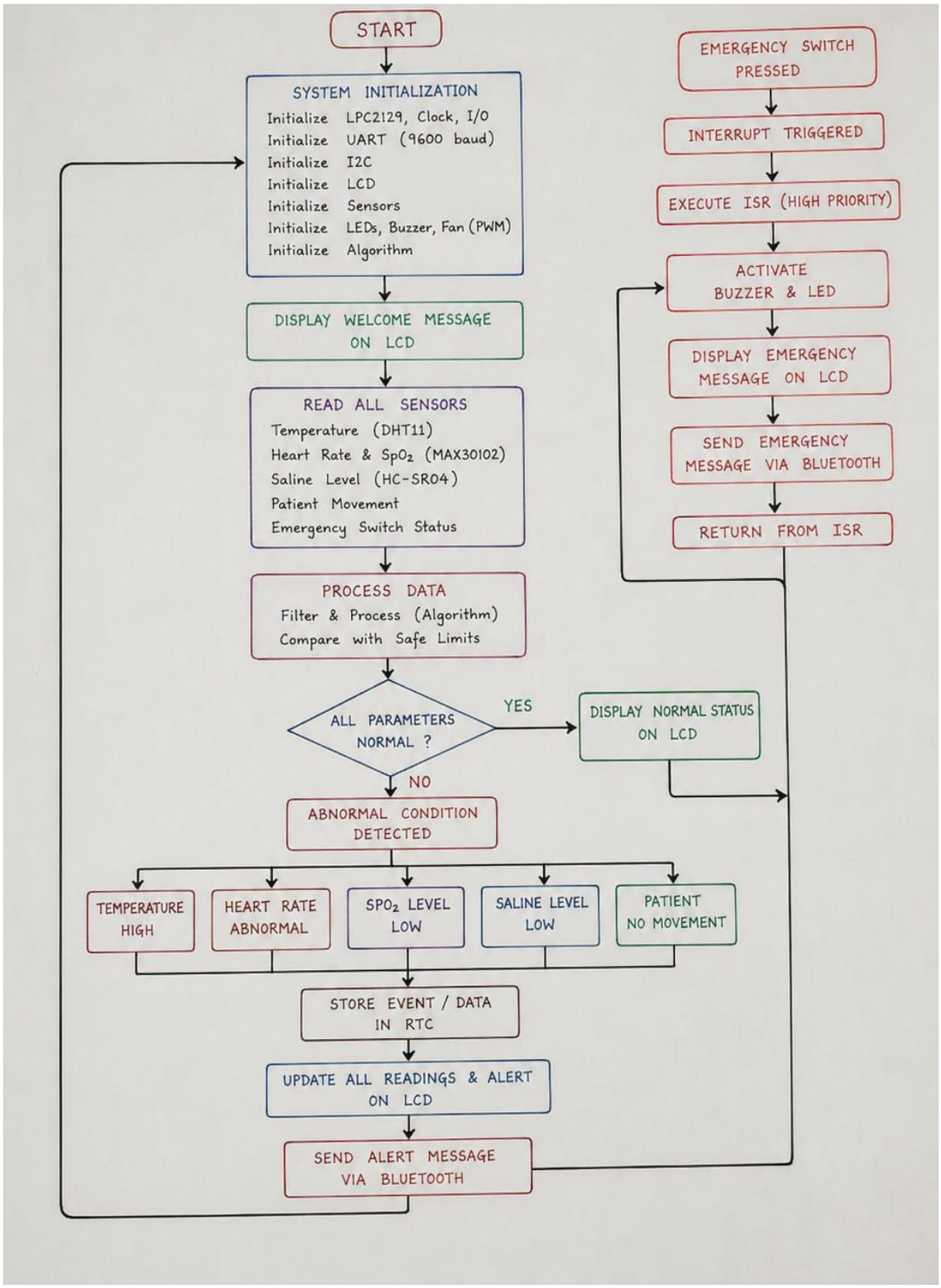

Fig. 3.2 – Working Flow of the Embedded Decision Logic

CHAPTER 4 | CODE REVIEW

4.1 Program Structure Overview

The firmware for the **Smart Healthcare Monitoring and Emergency Assistance System** is developed in **Embedded C** using the **Keil µVision IDE**. The program follows a modular structure in which each subsystem—system initialization, sensor data acquisition, health parameter processing, threshold evaluation, alert generation, LCD display, Bluetooth communication, and emergency handling—is implemented as an independent function. This modular approach improves readability, simplifies debugging, and allows future expansion of the system by integrating additional healthcare sensors or communication modules.

The main program continuously acquires patient health parameters from the connected sensors, processes the acquired data, compares it with predefined threshold values, and generates appropriate alerts whenever an abnormal condition is detected. The representative code modules presented below describe the software architecture adopted for the proposed system and form the foundation for the complete hardware implementation.

4.2 Key Code Modules

(a) **Main Program and Global Declarations**

```
#include <lpc21xx.h>

#include <stdio.h>

#include "header.h"

#define grn           (1<<16)

#define rd            (1<<17)

#define buz           (1<<18)

#define max_temp       38

#define min_heart_rate  60

#define max_heart_rate  99

#define min_spo2       90

#define ir_threshold    75000

MAX30102_DATA SAMPLE;

pulse_result PULSE;

int main(void)

{

   uart0_init(9600);
```

```
    adc_init();
    lcd_init();
    i2c_init();
    pulse_init();
    algorithm_init();
    while(1)
    {
      // Read sensors and monitor patient
    }
}
```

The main() function serves as the entry point of the patient health monitoring system. It initializes all the required peripherals, including the UART, ADC, LCD, I²C interface, and MAX30102 pulse sensor. After initialization, the controller continuously monitors the patient's physiological parameters by repeatedly reading the sensors and processing the acquired data. This simple loop ensures uninterrupted real-time patient monitoring.

(b) Pulse Sensor (MAX30102) Initialization

```
void pulse_init(void)
{
    Pulse_WriteReg(MAX_MODE_CONFIG,0x40);
    delay_ms(100);
    Pulse_WriteReg(MAX_FIFO_WR_PTR,0x00);
    Pulse_WriteReg(MAX_OVF_COUNTER,0x00);
    Pulse_WriteReg(MAX_FIFO_RD_PTR,0x00);
    Pulse_WriteReg(MAX_FIFO_CONFIG,0x0F);
    Pulse_WriteReg(MAX_SPO2_CONFIG,0x27);
    Pulse_WriteReg(MAX_MODE_CONFIG,0x03);
    Pulse_WriteReg(MAX_LED1_PA,0x24);
    Pulse_WriteReg(MAX_LED2_PA,0x24);
}
```

This function initializes the MAX30102 pulse oximeter sensor before measurements begin. The sensor is first reset, after which the FIFO pointers are cleared and the operating registers are configured. The LED drive current and SpO_2 measurement mode are also enabled to allow accurate acquisition of infrared and red light signals. Proper initialization is essential for obtaining reliable heart rate and blood oxygen saturation measurements.

(c) Pulse Sensor Data Acquisition

```
void Pulse_ReadFIFO(MAX30102_DATA *sample)
{
   unsigned char data[6];
   i2c_read_multi(MAX30102_ADDR,MAX_FIFO_DATA,data,6);
   sample->red=((unsigned long)data[0]<<16) )
            ((unsigned long)data[1]<<8) )
             data[2];
   sample->ir=((unsigned long)data[3]<<16) )
           ((unsigned long)data[4]<<8) )
            data[5];
}
```

The MAX30102 stores measured sensor samples in its internal FIFO memory. This function reads six bytes from the FIFO using the I^2C interface and reconstructs the 18-bit red and infrared ADC values. These raw optical signals are then supplied to the signal-processing algorithm, where they are used to calculate the patient's heart rate and blood oxygen saturation.

(d) Heart Rate and SpO_2 Processing

```
avg = moving_average(ir);
if(avg > prev_avg)
   rising = 1;
if((avg < prev_avg) && rising)
{
   result->bpm = (60 * SAMPLE_RATE) / diff;
}
```

The heart rate and SpO_2 algorithm processes the raw infrared and red LED signals obtained from the pulse sensor. A moving average filter is first applied to reduce noise in the infrared signal. Pulse peaks are detected from the filtered waveform, and the interval between successive peaks is used to calculate the heart rate in beats per minute (BPM). The ratio of the red and infrared signals is then used to estimate the blood oxygen saturation (SpO_2).

(e) Temperature Monitoring

```
cur_temp = adc_temp_read();

if(cur_temp > max_temp)

{

   IOSET0 = rd | buz;

   IOCLR0 = grn;

}
```

The body temperature is measured using the temperature sensor connected to the ADC channel of the LPC2148. The measured temperature is continuously compared with the predefined safe limit. If the temperature exceeds the threshold, the controller activates the warning LED and buzzer to alert the caregiver that the patient's body temperature is abnormal.

(f) Saline Level Monitoring

```
saline = adc_saline_read();

if(saline < saline_limit)

{

   uart0_str("Saline Level Low");

   IOSET0 = buz;

}
```

The saline level sensor continuously monitors the amount of intravenous (IV) fluid available in the saline bottle. When the saline level falls below the predefined threshold, the system immediately generates an alert through the buzzer and transmits a warning message via Bluetooth. This feature helps prevent interruption of IV fluid administration by notifying medical staff before the bottle becomes empty.

(g) LCD Display Module

```
lcd_cmd(0x80);

lcd_str("TEMP:");
```

```
lcd_float(cur_temp);

lcd_cmd(0xC0);

lcd_str("HR:");

lcd_int(PULSE.bpm);
```

The LCD module provides real-time visual feedback of the patient's health parameters. It displays the measured body temperature, heart rate, oxygen saturation level, and system status messages. Displaying the information locally enables healthcare personnel to observe the patient's condition without requiring additional monitoring equipment.

(h) Bluetooth Communication

```
sprintf(str,

"Temp=%.1f HR=%d SpO2=%.1f\r\n",

cur_temp,PULSE.bpm,PULSE.spo2);

uart0_str(str);
```

Bluetooth communication is implemented using the UART interface to transmit the patient's vital signs wirelessly to a mobile device or monitoring computer. The measured temperature, heart rate, SpO_2 value, and saline status are periodically sent as formatted serial data. This enables remote monitoring of the patient's health condition without requiring a wired connection.

(i) Buzzer and Alert System

```
if(cur_temp > max_temp ||
  PULSE.bpm < min_heart_rate ||
  PULSE.spo2 < min_spo2)
{
   IOSET0 = buz | rd;
   IOCLR0 = grn;
}
else
{
   IOCLR0 = buz | rd;
   IOSET0 = grn;
}
```

The alert module continuously compares the measured physiological parameters with predefined threshold values. Whenever the patient's temperature, heart rate, SpO_2 level, or saline level exceeds the safe operating limits, the controller activates the buzzer and warning LED to indicate an emergency condition. Under normal operating conditions, the green LED remains ON, indicating that all monitored parameters are within the acceptable range.

4.3 Code Analysis and Review Remarks

Review Aspect	Observation
Modularity	The project is organized into separate modules such as the main program, pulse sensor driver, I^2C driver, UART driver, LCD driver, and heart rate/SpO_2 algorithm. This modular design improves code reusability, readability, and maintenance.
Readability	The code uses meaningful function names, header files, macros, and comments, making it easier to understand and modify. The separation of hardware drivers from application logic further enhances readability.
Response Time	The system continuously acquires sensor data and processes it in real time. Patient parameters are updated with minimal delay, allowing quick detection of abnormal conditions.
Fault Priority	The program continuously compares measured values against predefined threshold limits. When abnormal temperature, heart rate, SpO_2, or saline level is detected, the system immediately activates the buzzer and warning LED while displaying the alert message.
Data Persistence	Logging to both RTC-time-stamped EEPROM and SD card ensures fault history survives a power interruption.
Scope for Improvement	Patient parameters are transmitted through Bluetooth/UART for external monitoring.
Scope for Improvement	The system can be enhanced by integrating Wi-Fi or GSM for cloud-based remote monitoring, adding EEPROM/SD card storage for patient history, improving the signal-processing algorithm, and incorporating additional biomedical sensors such as ECG or blood pressure sensors.

CHAPTER 5 | ADVANTAGES AND DISADVANTAGES

5.1 Advantages

- Provides **continuous real-time monitoring** of patient vital parameters.
- Simultaneously monitors **body temperature, heart rate, SpO_2, and saline level**.
- Generates **immediate alerts** through a buzzer and LED when abnormal conditions are detected.
- Displays patient health information clearly on the **LCD** for easy monitoring.
- Supports **wireless data transmission** via Bluetooth for remote observation.
- Uses the **MAX30102 sensor**, enabling accurate heart rate and blood oxygen saturation measurement.
- Low-cost and compact design suitable for hospitals, clinics, and home healthcare applications.
- Modular software structure makes the system **easy to maintain, debug, and upgrade**.
- Low power consumption, making it suitable for portable medical monitoring devices.
- Reduces the workload of healthcare professionals by automating patient monitoring.
- Enables early detection of critical health conditions, improving patient safety.
- Can be extended by integrating additional sensors or IoT cloud platforms for advanced healthcare monitoring. Relatively low hardware cost compared to commercial industrial SCADA/monitoring packages.

5.2 Disadvantages / Limitations

- Depends on proper placement of the **MAX30102 sensor**; incorrect finger positioning can affect heart rate and SpO_2 accuracy.
- Bluetooth communication has a **limited operating range**, making it unsuitable for long-distance monitoring without additional networking.
- The current system **does not store historical patient data** for future analysis.
- Accuracy may be affected by **patient movement** or external light interference during measurement.
- Only a limited number of vital parameters are monitored; it does not measure ECG, blood pressure, or respiration rate.
- Threshold values are **fixed** and require manual modification in the program for different patient conditions.
- The system relies on a **continuous power supply** and has no built-in battery backup.
- It is intended for **basic patient monitoring** and is not a replacement for certified medical diagnostic equipment.
- Sensor readings may vary due to environmental conditions or hardware calibration.
- The system currently supports **Bluetooth only** and lacks cloud-based or Internet-enabled remote monitoring.
- There is no user authentication or encryption for transmitted patient data, which may limit data security.
- Additional hardware is required to expand the system with more medical sensors or advanced monitoring features.

CHAPTER 6 | FUTURE SCOPE

While the current synopsis demonstrates a complete embedded safety-monitoring architecture, several enhancements can extend the system toward a production-ready industrial solution:

- Integrate **Wi-Fi or IoT cloud platforms** for real-time remote patient monitoring from anywhere.
- Develop a **mobile application** to display patient health parameters and instant alerts.
- Store patient records in a **cloud database or local memory (EEPROM/SD card)** for future analysis.
- Add **GPS and GSM modules** to send emergency alerts and the patient's location to caregivers or hospitals.
- Incorporate additional biomedical sensors such as **ECG, blood pressure, respiration rate, and glucose monitoring** for comprehensive health assessment.
- Implement **AI and machine learning algorithms** to predict potential health risks and detect abnormalities automatically.
- Improve the heart rate and SpO_2 processing algorithm for **higher accuracy under motion or noisy conditions**.
- Support **multiple patient monitoring** from a single central monitoring station.
- Develop a **battery-powered portable version** for home healthcare and ambulance applications.
- Add **voice alerts and multilingual support** to improve accessibility for patients and healthcare staff.
- Implement **secure data encryption and user authentication** to protect patient information during wireless transmission.
- Integrate the system with **hospital management systems (HMS)** or **electronic health records (EHR)** for seamless patient data management.
- Design a **wearable version** of the device for continuous long-term health monitoring.
- Include automatic **doctor and caregiver notifications** via SMS, email, or mobile app when abnormal vital signs are detected.
- Enhance the user interface with a **touchscreen display** and graphical visualization of patient health trends.

CHAPTER 7 | BIBLIOGRAPHY

The following references and resources were consulted for the design methodology, component selection and embedded programming concepts used in this synopsis report.

- Analog Devices, **"MAX30102 High-Sensitivity Pulse Oximeter and Heart-Rate Sensor for Wearable Health,"** Datasheet, 2018.
- Texas Instruments, **"LM35 Precision Centigrade Temperature Sensors,"** Datasheet, 2017.
- HC-05 Bluetooth Module, **"HC-05 Bluetooth Serial Module AT Command Set and User Manual,"** 2019.
- Hitachi Ltd., **"HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver Datasheet,"** 1999.
- Microchip Technology Inc., **"25LC1024 1-Mbit Serial EEPROM Datasheet,"** 2008.
- Joseph Yiu, **The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors**, 3rd ed., Newnes, 2014.
- Muhammad Ali Mazidi, Janice Gillispie Mazidi, and Rolin D. McKinlay, **The 8051 Microcontroller and Embedded Systems**, 2nd ed., Pearson Education, 2008.
- Raj Kamal, **Embedded Systems: Architecture, Programming and Design**, 3rd ed., McGraw-Hill Education, 2017.
- Simon Monk, **Programming Arduino: Getting Started with Sketches**, 2nd ed., McGraw-Hill Education, 2016.
- World Health Organization (WHO), **Medical Device Technical Series: Medical Device Regulations**, World Health Organization, Geneva, Switzerland.
- IEEE, **IEEE Standards Association**, "IEEE Standards for Medical Device Communication and Healthcare Monitoring Systems."
- Bluetooth SIG, **Bluetooth Core Specification Version 5.0**, Bluetooth Special Interest Group, 2016.
- Texas Instruments, **"Understanding Pulse Oximetry and Photoplethysmography (PPG),"** Application Report.
- Datasheets and technical documentation obtained from the respective manufacturers of the LPC2148 microcontroller, MAX30102 sensor, Bluetooth module, LCD display, and associated embedded system components.